CASE STUDY ON LOW-CODE LIMITATIONS

Student Admission Management System

1. Aim

To study the limitations encountered while developing a Student Admission Management System using a low-code development platform, and to evaluate how those limitations affect customization, workflow design, data handling, user experience, and long-term maintainability.

2. Theory

Low-code development platforms provide visual builders, drag-and-drop components, prebuilt forms, data tables, workflows, and integrations. They reduce the amount of hand-written code required and help teams build applications quickly. However, low-code applications still depend on the platform's available components, workflow rules, data model, licensing, performance limits, and integration options. When requirements become highly specific or the application grows, developers may need custom scripts, workarounds, or external services. These constraints can increase complexity and make migration to another platform difficult.

3. Case Study Scenario

A college team created a Student Admission Management System on a low-code platform. The application was designed to centralize student profiles, eligibility checks, application documents, admission criteria, application statuses, and fee payments. The dashboard displayed totals and charts, while forms and workflows supported the admission process. The supplied screenshots show the dashboard, eligibility-check report, application-status board, admission-criteria table, and a workflow configured to validate eligibility marks.

4. System Features Implemented

- Dashboard with total amount paid, eligibility checks, application documents, student profiles, and academic-year counts.
- Eligibility-check records showing student IDs, criteria codes, check dates, marks, pass/fail status, remarks, and verifier.
- Application status board with columns such as Submitted, Under Review, Admitted, and Rejected.
- Admission criteria table with criterion code, subject, minimum marks, required documents, mandatory flag, and description.
- Form workflow named “Validate Eligibility Marks” to apply a validation action during submission.

5. Identified Limitations During Implementation

- **Customization constraints:** The visual builder offers predefined layouts and controls. Complex page layouts or highly specific interactions may require workarounds or custom code.
- **Workflow complexity:** The eligibility workflow is useful for basic validation, but complex rules involving multiple criteria, exceptions, and approval paths can become difficult to configure and test.
- **Data relationships and validation:** Student IDs and criteria codes must remain consistent across forms and reports. Incomplete validation or poorly defined relationships can produce confusing or duplicated records.
- **Reporting and dashboard limits:** Built-in charts and tables are convenient, but advanced filtering, custom calculations, and detailed exports may be limited by the platform.
- **Performance and scalability:** As student records, uploaded documents, and workflow actions increase, page loading and report generation may slow down depending on platform capacity and data design.
- **Integration and portability:** Connecting external college systems or moving the application to another platform may require additional configuration, APIs, or data transformation.

- **Vendor dependency and cost:** The application depends on the platform's availability, pricing, feature tiers, and product roadmap. Trial or plan limits can restrict deployment and continued use.
- **Testing and debugging:** Visual workflows can hide execution details, making it harder to trace errors than in a fully code-based application.

6. Mitigation and Recommendations

- Define the admission data model carefully and use consistent unique student identifiers.
- Break workflows into small, named steps and test normal cases, missing data, invalid marks, and exception scenarios.
- Document business rules, permissions, field validations, and dependencies.
- Use role-based access to protect student information and payment records.
- Monitor performance as records grow and archive or paginate large datasets where supported.
- Confirm platform licensing, export options, API access, and backup procedures before production deployment.

SCREENSHOTS OF THE IMPLEMENTED SYSTEM

The following screenshots document the main modules and workflow used in the case study.

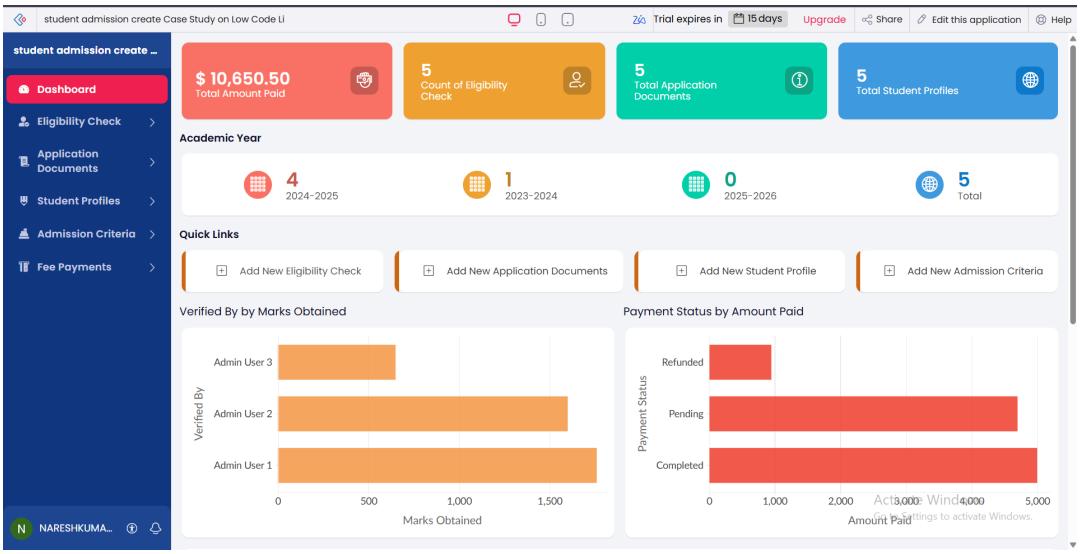


Figure 1. Dashboard showing admission-system summary cards, academic-year counts, quick links, and charts.

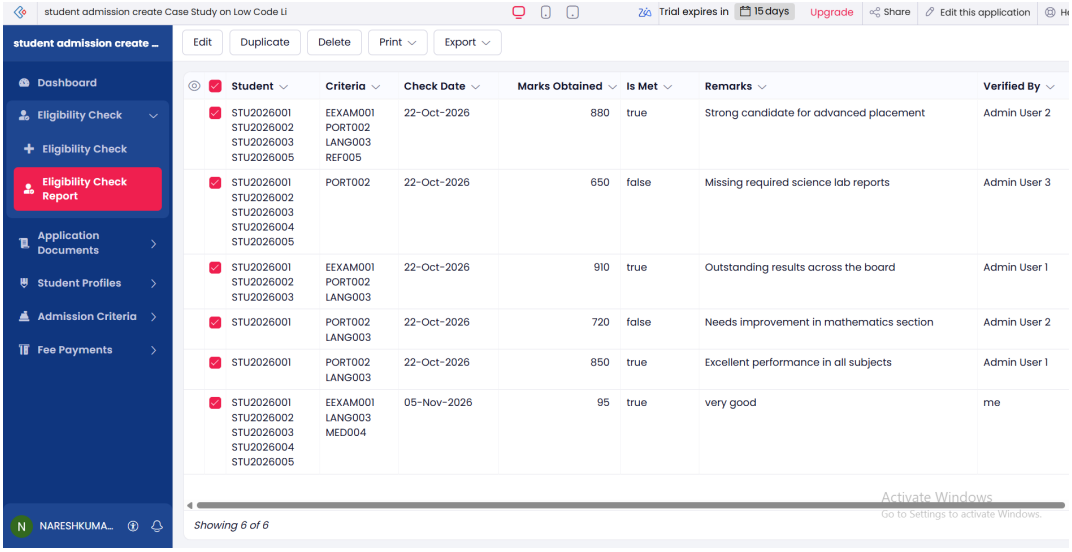


Figure 2. Eligibility Check Report showing student records, criteria, marks, eligibility status, remarks, and verification details.

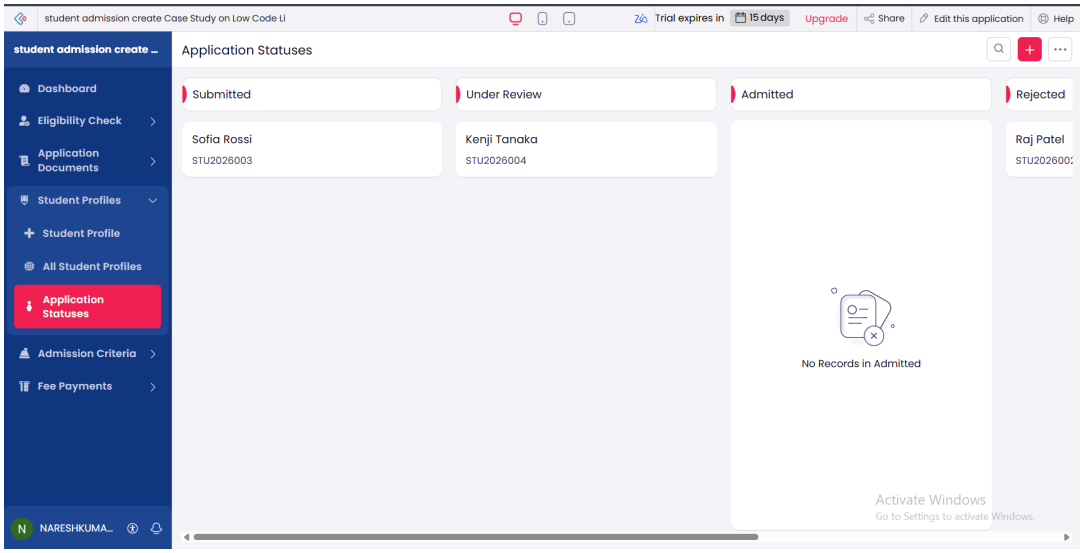


Figure 3. Application Statuses board showing applications grouped by submission and review status.

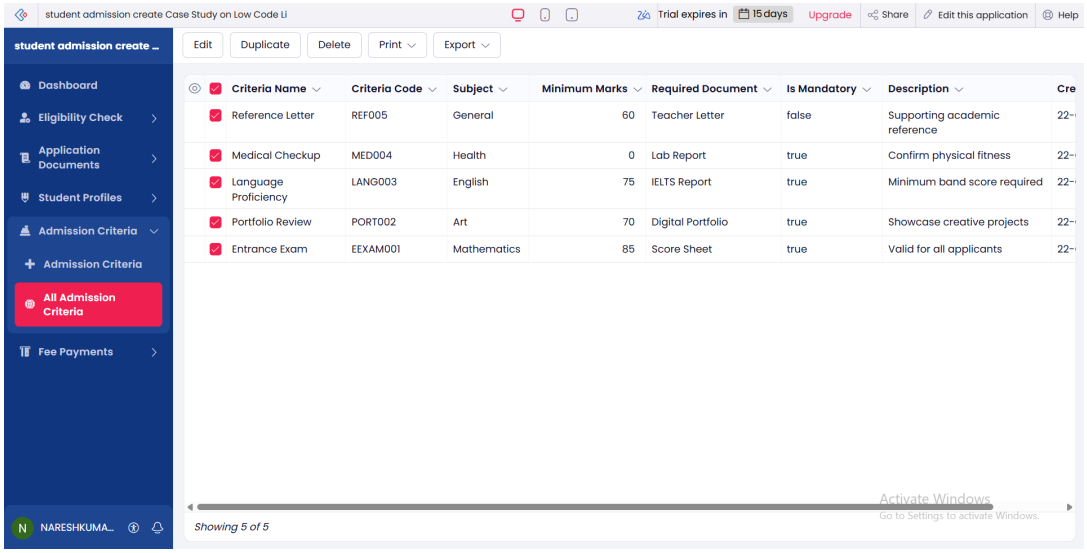


Figure 4. Admission Criteria table showing required marks, documents, mandatory fields, and descriptions.

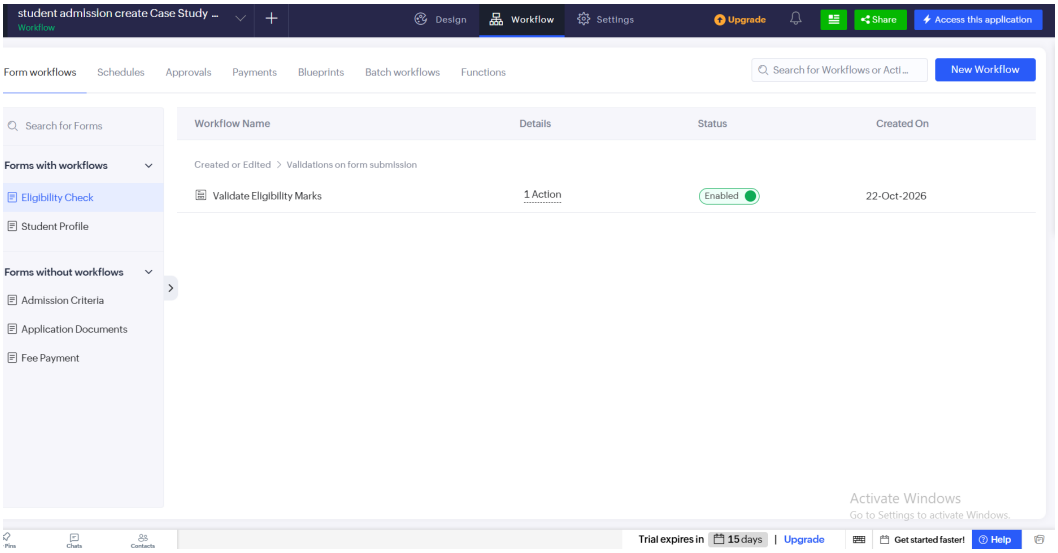


Figure 5. Workflow configuration showing the “Validate Eligibility Marks” form workflow.

7. Result

The low-code platform enabled the team to assemble a working prototype of the Student Admission Management System quickly, with a dashboard, admission-related records, status tracking, and a basic eligibility validation workflow. It reduced initial development effort and made the application easier to demonstrate. The case study also showed that low-code is not limitation-free: complex rules, deeper customization, advanced reporting, scalability, integration, and platform dependency require careful planning. Therefore, low-code is suitable for rapid prototyping and standard administrative workflows, while systems with complex or high-volume requirements should be evaluated for hybrid or custom-code components.